

Investigación Parte A (PEP 8) PEP 8 (Python Enhancement Proposal 8) se refiere a una guía de convenciones estilísticas a seguir para escribir código en python, que van desde conocer el tamaño máximo para las líneas de código, propuestas en cómo nombrar a las variables, volver el código legible, entre otras cosas. Incluso existen linters (herramientas que analizan el código de forma automática para detectar errores o malas prácticas) y auto formatters (herramientas que pueden reescribir código o indicar nuestros errores) que podemos usar en caso de no recordar todas las convenciones al escribir código. A continuación colocamos 10 ejemplos donde se puedan visualizar las aplicaciones de estas convenciones al código: Longitud máxima de línea (79 caracteres) Limitar todas las líneas de código a un máximo de 79 caracteres para evitar el desplazamiento horizontal y permitir la lectura simultánea de múltiples archivos. Incorrecto: `def calcular_prima(suma_segurada, factor_edad, descuento_no_fumador, recargo_extra):`
`return(suma_segurada*factor_edad/1000)-descuento_no_fumador+recargo_extra` Correcto : `def calcular_prima(suma_segurada, factor_edad, descuento_no_fumador, recargo_extra):`
`prima_base = (suma_segurada*factor_edad)/1000`
`return prima_base - descuento_no_fumador +`
`recargo_extra` Líneas en blanco para separación de estructuras Principio : Separar funciones globales y clases de ni

Incorrecto:

```
class Asegurado:
    def __init__(self, edad):
        self.edad = edad
    def es_mayor(self):
        return self.edad >= 18
def funcion_auxiliar():
    pass
```

Correcto:

```
class Asegurado:
    def __init__(self, edad):
        self.edad = edad

    def es_mayor(self):
        return self.edad >= 18

def funcion_auxiliar():
    pass
```

Organización y orden de importaciones

Principio: Cada import debe residir en una línea independiente al inicio del archivo, agrupados en tres bloques: biblioteca estándar, librerías de terceros y módulos locales.

■ *Incorrecto:*

```
import sys, os, math
from aseguradora.modelos import Asegurado
```

■ *Correcto:*

```
import os
import sys

from aseguradora.modelos import Asegurado
```

Espacios en blanco en expresiones y sentencias

Principio: Rodear operadores binarios (asignación, aritméticos, relacionales) con un espacio a cada lado, pero evitar espacios inmediatamente dentro de paréntesis, corchetes o llaves.

■ *Incorrecto:*

```
x=5
resultado = ( x * 2 ) + [ 1, 2, 3 ][ 0 ]
```

■ *Correcto:*

```
x = 5
resultado = (x * 2) + [1, 2, 3][0]
```

Convenciones de nomenclatura (*Naming Conventions*)

Principio: Utilizar `snake_case` para funciones y variables, `PascalCase` (o *CapWords*) para nombres de clases, y `UPPER_SNAKE_CASE` para constantes globales.

■ *Incorrecto:*

```
tasa_cambio = 21.13
class factor_edad:
    def CalcularFactor(self, EdadAsegurado):
        pass
```

■ *Correcto:*

```
TASA_CAMBIO = 21.13

class FactorEdad:
    def calcular_factor(self, edad_asegurado):
        pass
```

Comparaciones de identidad con None

Principio: Las comparaciones con valores únicos singleton como None deben efectuarse siempre empleando operadores de identidad (`is` o `is not`), jamás mediante igualdad de valor (`==`).

■ *Incorrecto:*

```
if valor == None:
    inicializar_variable()
```

■ *Correcto:*

```
if valor is None:
    inicializar_variable()
```

Comprobaciones implícitas de verdad booleana

Principio: No comparar directamente secuencias vacías o valores booleanos contra `True` o `False`; evaluar la secuencia directamente por su valor de verdad intrínseco.

■ *Incorrecto:*

```
if len(lista_palabras) == 0:
    return None
if es_fumador == True:
    aplicar_recargo()
```

■ *Correcto:*

```
if not lista_palabras:
    return None
if es_fumador:
    aplicar_recargo()
```

Manejo explícito de excepciones

Principio: Evitar el uso de cláusulas `except:` vacías que atrapen `BaseException` (incluyendo señales de terminación del sistema). Especificar la excepción concreta esperada.

■ *Incorrecto:*

```
try:
    edad = int(input("Edad: "))
except:
    print("Error al procesar.")
```

■ *Correcto:*

```
try:
    edad = int(input("Edad: "))
except ValueError:
    print("Error: Ingrese un número entero válido.")
```

Espacios alrededor de argumentos por defecto

Principio: No utilizar espacios alrededor del signo = cuando se definan valores por defecto en argumentos de funciones o al pasar argumentos por palabra clave (*kwargs*).

■ *Incorrecto:*

```
def normalizar_texto(texto, remover_acentos = True):
    return texto.strip()
```

■ *Correcto:*

```
def normalizar_texto(texto, remover_acentos=True):
    return texto.strip()
```

0.1. Parte B – Principios de Diseño Orientado a Objetos (SOLID)

0.2. Referencias Consultadas